

how would you defined agentic memory?

Thought for 16s >

Here is a tight definition and a practical breakdown.

What it is

Agentic memory is the structured, policy-driven store of information that an autonomous agent writes to, retrieves from, and prunes over time to improve decisions across tasks. It is not a raw log or a static KB. It is a selective, credited, and revisable substrate for behavior.

Formally, let memory be $M = \langle \mathcal{I}, \pi_{\text{write}}, \pi_{\text{retrieve}}, \pi_{\text{consolidate}}, \pi_{\text{forget}} \rangle$ where $\mathcal{I}$ is a set of items with provenance, confidence, and scope. Policies control what gets stored, when it is recalled, how it is summarized, and what is discarded.

Types

- Working memory – short-lived scratchpad for the current deliberation.
- Episodic memory – timestamped observations, actions, outcomes, and rewards.
- Semantic memory – stable facts, user preferences, tools, constraints.
- Procedural memory – skills, plans, macros, heuristics, and learned controllers.
- Meta-memory – statistics about what to store, when to fetch, and what to forget.

Minimum viable data model

Each item should have:

- id, type, content, source, timestamp
- confidence, utility score, lineage links, scope owner, TTL
- indices: symbolic keys, vector embedding, and graph edges

Core mechanics

- Write policy – gate on novelty, prediction error, or utility gain; dedupe by semantic similarity and key fields.
- Retrieval policy – hybrid search that uses task intent, symbols, and vectors; rank by expected utility, not cosine alone.

- Consolidation – periodic summarization and schema folding to reduce entropy while keeping causal links.
- Forgetting – TTLs, decay schedules, rate limits, and size caps with eviction that respects provenance and utility.
- Credit assignment – link items to outcomes so retrieval and write thresholds learn over time.

How it differs from adjacent things

- Context cache – transient. Agentic memory persists with policies and credit.
- Vector DB – an index, not a memory. Memory uses vectors plus symbols, graphs, and policies.
- Knowledge base – curated and static. Agentic memory is operational and self-modifying.

Operational loop

1. Observe → encode → candidate items
2. Decide with retrieval selectors conditioned on goal and state
3. Act → measure outcome → assign credit
4. Consolidate and prune on a schedule

Quality and safety bar

- Utility lift on target tasks, retrieval precision@k, false recall rate, latency, and bloat ratio
- Provenance required for every item, scoped tenancy per user, reversible edits, explicit consent for personal data
- Red team prompts that probe for leakage or spurious personalization

Anti-patterns

- Hoarding everything or summarizing without sources
- Vector-only recall for symbolic constraints
- Personalization without scope and consent
- Static memory that never learns write and fetch thresholds

Tiny example schema

json

```
{
  "id": "m_4821",
  "type": "preference",
  "content": "User prefers coil springs over air suspension",
  "source": "chat:2025-09-28",
  "timestamp": "2025-09-28T10:32:00Z",
  "confidence": 0.92,
  "utility": 0.77,
  "scope": "user:rand",
  "ttl": "365d",
  "lineage": ["chat:2025-09-28#msg12"],
  "keys": ["suspension", "preference"],
  "embedding": "...",
  "edges": [{"rel": "contrasts_with", "to": "air_suspension"}]}
}
```

Litmus test

If removing the memory does not measurably change decisions or outcomes, it was not agentic memory. If it changes them in ways you can attribute, monitor, and revise, it is.



i view agentic memory as closely related to optimized context generation. agents need to be able to remember and recall salient things, but not in a vacuum. they must do so in a way that maximizes their efficacy in their tasks, which means identifying the right (most relevant, actionable, useful) information to recall, and balancing the payload that gets delivered into the context window for an ideal balance of efficacy and efficiency.

given that the context being optimized may come from "memory" as well as from various data resources, queries against tools, other agents, etc, how can we think about a unified model?

Here is a clean way to unify “memory” with on-demand retrieval and tool use so the agent always assembles the right context under real budgets.

Unifying idea

Treat everything the agent might put into the prompt as a **context candidate** with the same schema and the same selection logic, regardless of origin: long-term memory, prior episodes, live tool queries, files, other agents, or cached summaries. Selection then becomes a budgeted optimization problem over these candidates.

Canonical schema

Each candidate is a record with features that support scoring, constraints, and learning.

json

 Copy code

```
{
  "id": "src:memory:m_4821",
  "kind": "preference | fact | episode | plan | tool_result | file_chunk |",
  "content": "string or structured payload or lambda for deferred exec",
  "source": "memory|tool:serp|tool:db|agent:critic|file:path",
  "timestamp": "ISO8601",
  "features": {
    "relevance": 0.0,
    "actionability": 0.0,
    "freshness": 0.0,
    "reliability": 0.0,
    "novelty": 0.0,
    "sensitivity": 0.0
  },
  "costs": {
    "tokens_est": 180,
    "latency_est_ms": 120,
    "dollars_est": 0.0006
  },
  "risks": {
    "leakage": 0.1,
    "policy": ["pii:low"]
  },
  "links": [{"rel": "supports", "to": "task:goal#A"}, {"rel": "derived_fro",
  "ttl": "365d",
```

```
"scope": "user:rand"
}
```

Notes

- `deferred_query` means “plan to fetch,” not yet executed. It carries an estimator of expected value and cost if executed.
- Memory items are just another `kind`, not a special path.

Objective

For a task state x with token budget B and latency budget L , choose a subset S of candidates (including executing some deferred queries) to maximize expected task utility.

A practical surrogate is a submodular knapsack:

$$\max_{S \subseteq \mathcal{C}} J(S) = \sum_{i \in S} u_i - \lambda \sum_{i \in S} \text{redundancy}(i \mid S \setminus \{i\})$$

subject to

$$\sum_{i \in S} \text{tokens}_i \leq B, \quad \sum_{i \in S} \text{latency}_i \leq L, \quad \text{risk}(S) \leq \rho$$

with

$$u_i = w_r \text{relevance}_i + w_a \text{actionability}_i + w_f \text{freshness}_i + w_\rho \text{reliability}_i - w_c \text{cost}_i -$$

This gives a principled greedy selector that balances “how good” with “how much” and “how risky.”

System components

1. Task specifier

Normalize the goal into a typed need profile: intent, constraints, required tools, allowed data scopes, budgets.

2. Candidate generators

- Memory retrievers: episodic, semantic, procedural.
- Index retrievers: vector + symbolic + graph.
- Tool planners: produce `deferred_query` candidates with EV and cost.
- Cohort agents: proposals plus self-scores.

3. Budgeter

Set B, L, ρ per task and user policy. Reserve headroom for the model's own reasoning.

4. Selector

Hybrid MMR + knapsack greedy:

- Stage A: select a small set of deferred queries with the best EV per unit cost. Execute them.
- Stage B: merge new results, then run submodular greedy to pick the final context set.

5. Composer

Populate a **context blueprint** with typed slots rather than dumping a flat blob.

json

 Copy code

```
{
  "slots": [
    {"name": "task_header", "max_tokens": 200},
    {"name": "ground_truth_facts", "max_tokens": 400},
    {"name": "user_prefs", "max_tokens": 150},
    {"name": "constraints_and_apis", "max_tokens": 250},
    {"name": "retrieved_evidence", "max_tokens": 500},
    {"name": "citations", "max_tokens": 100}
  ],
  "invariants": [
    "no PII beyond scope: user:rand",
    "include provenance for every non-obvious claim",
    "compress episodes older than 30d unless requested"
  ]
}
```

Fill with compression operators: merge, summarize, distill, cite, dedupe. Respect slot caps.

6. Critic and guardrails

Check for leakage, stale facts, contradiction, and policy violations. If violated, evict or compress; never silently exceed budgets.

7. Learner

Update weights and thresholds from outcomes. Use counterfactual estimators over logs to learn better EV for deferred queries and better slot allocations.

Retrieval and scoring details

- **Relevance:** intent-conditioned hybrid search. Score with a small cross-encoder, not cosine alone.
- **Actionability:** does this item change decisions or enable a concrete step. Learn via credit assignment to items that were present in successful runs.
- **Reliability:** provenance prior times recency calibration. Penalize ungrounded summaries unless citations are present.
- **Freshness:** time decay with domain specific half-life.
- **Novelty:** MMR diversity to avoid near-duplicates.
- **Sensitivity and scope:** per-user tenancy, consent flags, and policy labels feed the risk constraint.

Pseudocode for the selector

python

 Copy code

```
def select_context(task, generators, budget):
    C = []
    for g in generators:
        C += g.propose(task)          # includes memory, indices, d

    # Stage A: pick and execute a few high-EV deferred queries
    Q = [c for c in C if c.kind == "deferred_query"]
    Q_sorted = sorted(Q, key=lambda q: q.ev / (q.costs["latency_est_ms"])+q
    execd = []
    for q in Q_sorted:
        if budget.latency_left() >= q.costs["latency_est_ms"]:
            execd += execute(q)
            budget.consume(latency=q.costs["latency_est_ms"])
    C += execd

    # Stage B: submodular greedy under token budget and risk cap
    S = []
    while True:
        best = argmax(
            [marginal_gain(i, S, task) / token_cost(i) for i in C if feasi
            )
        if best is None: break
        S.append(best); budget.consume(tokens=token_cost(best))
```

```
C.remove(best)
return compose(S, task.blueprint)
```

Why this answers your “unified” requirement

- Memory and live data are first-class citizens with the same scoring, budgeting, and risk treatment.
- The agent picks not just what to include, but which queries to run at all, based on expected value.
- Context is structured into slots that the model can exploit, which improves efficacy per token.
- The policy learns. Items that correlate with wins get higher expected value; noisy items decay.

Operational metrics

- **Efficacy per token:** delta task success or reward per 1k tokens added to context.
- **Precision@k of retrieval** and **false recall rate**.
- **Redundancy ratio:** selected tokens that add no new unique facts.
- **Staleness rate:** fraction of facts past their freshness half-life.
- **Leakage incidents** and scope violations.

Anti-patterns to avoid

- Treating the vector store as the memory rather than an index.
- Executing all queries before deciding if their results fit the budget.
- Unstructured concatenation of everything into the prompt.
- Summarizing without citations or provenance.

Minimal worked example

Goal: “Plan coil-spring conversion options for a 2017 Cayenne S under Denver emissions.”

- Generators propose:
 - a) semantic memory items for user suspension preferences, emissions constraints.
 - b) file chunks from prior build notes.
 - c) deferred queries to two shops and one parts catalog.
 - d) an episodic memory from a prior chat with costs and risks.

- Selector executes one parts query with highest EV per ms, skips the other two due to budget.
- Composer fills slots: constraints, user prefs, retrieved evidence with citations, and a short episodic note.
- Critic flags an old note about AHC that does not apply, which is evicted.
- Learner credits the parts catalog retrieval and the “preference” memory for improving plan quality, increasing their future weights.

This gives you a single, coherent mechanism that optimizes what the agent remembers, what it fetches, and what it finally says, all under the same objective and budget.



i want you to deeply consider the following bits of research, search for additional relevant research, and provide a well-considered update on how to design an agentic memory system that can work with extremely high efficacy and efficiency in an environment where one or more agents are collaborating on tasks, have access to shared memory and other data resources, access to tools, etc.

1. <https://arxiv.org/abs/2502.12110v11>
2. <https://arxiv.org/abs/2510.04618>
3. <https://voyager.minedojo.org/>
4. <https://abehrouz.github.io/files/NL.pdf>

Thought for 55s >

Here is a distilled, research-grounded design you can actually build. I fold in your four sources, add adjacent work where it materially changes the design, and call out tradeoffs.

What the new papers add

- **A-MEM** proposes dynamic, agent-driven memory construction and linking, inspired by Zettelkasten. It treats each memory as a rich note, automatically creates links,

and allows new items to update old ones. Net effect: a living graph, not a static log.

arXiv +1

- **Agentic Context Engineering** frames context as an evolving artifact. It introduces mechanisms for self-improvement by curating and rewriting context over time without retraining model weights. Treat the context as a learned policy object, not a passive buffer. arXiv +2
- **Voyager** shows how an embodied agent accrues skills into a library, with iterative self-verification, avoiding catastrophic forgetting and compounding capabilities. The transferable insight: persistent, interpretable skills plus automatic curriculum. Voyager +1
- **Nested Learning (NL)** argues for multi-timescale updates across components. Read it as a design cue for memory tiers that update at different frequencies, with explicit context flows and local learning objectives. Ali Behrouz

Strong adjacent prior art you should pull in:

- **Generative Agents** memory scoring via relevance, recency, importance. Good baseline, but extend it. ACM Digital Library +1
- **GraphRAG** for global structure and community summaries, but know its incremental-update pain. Use with care. arXiv +2
- **Zep** for a temporal knowledge graph and bitemporal facts that outperform MemGPT on deep memory retrieval and temporal reasoning. Use its ideas even if you do not adopt the product. arXiv +2
- **MemoryOS** for a systems view of memory as tiers with OS-like APIs. Useful framing for interfaces and isolation. ACL Anthology

System architecture for collaborative agentic memory

1) Unified item model

Treat anything that could be injected into context as a **Context Candidate** with one schema, regardless of origin.

yaml

 Copy code

```
id, kind, content, source, timestamp
features: relevance, actionability, freshness, reliability, importance
costs: tokens_est, latency_est_ms, dollars_est
risks: sensitivity_label, leakage_prob
```

```

provenance: [citations], lineage
scope: {tenant, user, channel}
graph: {entities, relations, time_valid: [t_start, t_end]}
status: {raw|summarized|refined|skill}

```

Kinds include: memory_note (A-MEM style), tool_result, file_chunk, episode, plan, constraint, policy, skill, and deferred_query. This gives one scoring and budgeting surface for memory, tools, and data.

2) Two-level store with a temporal graph in the middle

- **Per-agent private store**

Fast KV + vectors for working set, plus a compact episodic log. Short TTL, aggressive dedupe.

- **Shared memory service**

A **bitemporal knowledge graph** that merges: episodic events, facts, tool outputs, and skills. Maintain valid-time and transaction-time, plus provenance. Use streaming subgraph updates and incremental community summaries to dodge GraphRAG rebuild costs. [arXiv +2](#)

Implementation sketch

- Storage: property graph with temporal edges, vector index for recall, object store for raw payloads.
- Write path: A-MEM's linkage and evolution rules create backlinks and trigger selective updates on impacted nodes. [arXiv](#)
- Update cadence: adopt NL's multi-timescale policy
 - Fast path: append episodes, tool results, scratch notes.
 - Medium path: nightly consolidation and note evolution.
 - Slow path: skill distillation and community summaries. [Ali Behrouz](#)

3) Skill library as first-class memory

Promote stable procedures into named **skills** with tests and descriptors, a la Voyager. Index skills by preconditions, postconditions, and resources touched. Persist metrics like historical win rate and cost. [Voyager](#)

4) Context assembly as a budgeted optimization

Selection is a knapsack over candidates under token, latency, and risk budgets. Start from Generative Agents' recency–relevance–importance, then extend:

$$u_i = w_r \text{relevance}_i + w_a \text{actionability}_i + w_f \text{freshness}_i + w_{imp} \text{importance}_i + w_g \text{gr}$$

Use submodular greedy with an MMR-style redundancy penalty, and a separate pass to choose which **deferred queries** to execute by expected value per unit latency.

ACM Digital Library

5) Evolving context loop

Adopt ACE's insight: after each task, rewrite context artifacts and memory. Concretely: summarize outcomes, revise facts in the temporal graph, promote or demote skills, and update per-agent retrieval weights. This is learning without weight updates. [arXiv](#)

6) Collaboration model

- **Blackboard pattern:** the shared temporal graph is the blackboard. Agents read, write, and subscribe to subgraph changes. Use CRDTs or append-only ledgers for conflict resolution, then consolidate in slow path. [arXiv +1](#)
- **Scopes:** tenant → project → role → user. Agents carry scoped capabilities. Private notes can be cited into shared summaries via redacted views.
- **Roles:** researcher, planner, executor, critic, memory-keeper. The memory-keeper enforces write policies and runs consolidation jobs.

7) Write, retrieval, and evolution policies

Write gates

- Novelty threshold on symbolic keys and embeddings.
- Prediction error or outcome delta triggers a write.
- A-MEM evolution: when a new note lands, update links and possibly attributes of related notes. Keep a change-of-belief trail. [OpenReview](#)

Retrieval

- Hybrid: lexical, vector, and **graph-aware**. Start from candidate neighborhoods in the temporal graph, then re-rank with a cross-encoder. Include community summaries for global context when local evidence is sparse. [arXiv](#)

Evolution

- ACE rule: for recurring tasks, learn a context blueprint with slot caps and canonical snippets. Periodically refactor long chains of episodes into skills or policies. arXiv

8) Safety and governance

- Every item carries provenance and time validity. Selection enforces a risk budget. Sensitive scopes cannot cross to shared memory without a redaction transform.
- Use MemoryOS-style APIs to separate memory operations from agent prompts, making audit and rollback straightforward. ACL Anthology

Minimal execution blueprint

Services

1. Ingestor: turns episodes, tool results, and chat into candidate items.
2. Linker: A-MEM style linking, dedupe, evolution. arXiv
3. Graph store: temporal KG with streaming subgraph updates and per-community summaries. arXiv +1
4. Retriever: hybrid search over text, vectors, and graph neighborhoods.
5. Selector: knapsack + MMR with EV-based query planner.
6. Composer: slot-based blueprints with compression operators.
7. Learner: updates weights, slot caps, and write thresholds based on outcomes.
8. Skill forge: promotes stable plans into skills with tests, metrics, and interfaces.

Voyager

Data paths

- Fast path: append to private store, candidate linking, immediate retrieval.
- Medium path: shared graph write, neighborhood refresh, summary refresh.
- Slow path: community summaries, skill distillation, policy learning.

Critical tradeoffs and how to handle them

- **GraphRAG cost vs benefit.** Global graphs are expensive and historically non-incremental. Solution: stream updates and limit graph work to neighborhoods and affected communities. Keep global summaries on a slow cadence. Reddit +1
- **Vector-only recall misses structure.** Use vectors to shortlist, but lean on graph neighborhoods and provenance for final ranking. arXiv
- **Catastrophic forgetting.** Voyager's library and NL's multi-timescale updates mitigate this. Do not overwrite skills based on single failures. Use rolling win-rate

thresholds. Voyager +1

- **Latency budgets.** Plan deferred queries by expected value per ms. Enforce a reserved token headroom for the model's own reasoning.
- **Multi-agent contention.** CRDT or ledger first, human-readable consolidations later. Enforce role-based write policies and redaction.

Metrics you should track

- Efficacy per 1k tokens added to context.
- Precision@k and false recall rate for retrieval.
- Redundancy ratio in assembled contexts.
- Temporal correctness rate on queries requiring time validity. arXiv
- Skill ROI: win rate, avg cost, reuse count. Voyager
- Graph maintenance cost per hour and per update. arXiv

A concrete example

Task: two agents write and ship a service.

- Researcher proposes deferred queries to API docs and prior incidents. Planner proposes 3 skills to reuse.
- Selector executes one docs query and one incidents query with best EV per ms.
- Retriever pulls a small neighborhood around "rate limiting" and its community summary.
- Composer fills a 6-slot blueprint, including constraints and two short skill cards.
- After deployment, critic logs an outage fix. Linker updates the "retry policy" note and bumps the win rate on the selected skill. ACE routine revises the canonical context snippet for future runs. arXiv

Why this design works

- It unifies memory, tools, files, and skills as context candidates under one optimization surface.
- It borrows A-MEM's dynamic linking, ACE's evolving context, Voyager's skill accrual, and NL's multi-timescale scheduler, then grounds retrieval with a temporal graph rather than a bag of chunks. arXiv +3

If you want, I can turn this into a compact spec with APIs and a reference implementation plan for your stack.



First, research DSPy. I think that its concepts of signatures and optimizers might provide real benefits when AI use happens within this system (not by the agents interacting with this system, which might be Claude Code, Codex, Gemini, or others).

Once you've researched and understood that, provide me with a detailed spec for use by an agentic coding system to build this. Make sure the spec also includes research, reference plans, and whatever else would be needed to make it highly effective, while also decomposed enough to be context efficient.

Thought for 56s >

Below is a research-grounded, build-ready spec that treats DSPy as the “inside the system” LM layer for optimizing memory write/retrieval/summarize/compose operations. It is decomposed so an agentic coding system can implement it incrementally without ballooning context.

Research takeaways on DSPy

- **Signatures** – declarative I/O contracts for LM steps. You tell the LM what to produce, not how to prompt it. DSPy
- **Modules** – implementations of signatures like `dspy.Predict`, `dspy.ChainOfThought`, etc., that turn signatures into runnable blocks. DSPy
- **Optimizers** (formerly teleprompters) – algorithms that compile your program to effective prompts and, in some cases, weights, by maximizing a metric on a trainset. Key ones for us: **MIPROv2** for joint instruction + few-shot optimization, **COPRO** for instruction optimization, and **BootstrapFewShotWithRandomSearch** for example selection. DSPy +3
- **Results and framing** – DSPy compiles LM pipelines into self-improving programs; the ICLR paper and docs emphasize replacing brittle prompt strings with trainable, metric-driven components. arXiv +2

Implication for our system: treat every LM-touching operation in memory as a DSPy subprogram with a signature, a module, a local metric, and an optimizer. That gives us measurable, auto-tuned behavior rather than ad-hoc prompts.

System goal

Design an **agentic memory service** for multi-agent collaboration that picks, shapes, and composes context with very high efficacy per token and per millisecond. The service exposes deterministic APIs to agents, while internally using DSPy to learn optimal instructions, examples, and lightweight finetunes for LM-dependent subroutines.

High-level architecture

1. Stores

- Private per-agent store – KV + vector for working set and a short episodic log.
- Shared temporal knowledge graph – facts, episodes, skills, provenance, valid-time and transaction-time.
- Artifact store – raw payloads and traces.

2. Core services

- **Ingest & Link** – canonicalize events into memory items, link and dedupe.
- **Candidate Generator** – memory recall, index recall, deferred query plans, prior summaries.
- **Selector** – knapsack + MMR under token, latency, and risk budgets.
- **Composer** – fills a slot-based blueprint with compressed, cited content.
- **Governance** – redaction, scope, consent, PII policy.
- **Learner** – updates thresholds and weights from outcomes.
- **DSPy LM layer** – houses all LM-touching subroutines as DSPy programs.

3. External agents

Claude Code, Codex, Gemini, etc., talk to the memory service over stable API contracts. They do not see DSPy internals.

Unified item model

Every candidate for the context window shares one schema.

json

 Copy code

```
{
  "id": "src:memory:m_4821",
  "kind": "memory_note | tool_result | file_chunk | episode | plan | const",
  "content": {...},
  "source": "memory|tool:...|file:...|agent:...",
  "timestamp": "ISO8601",
  "features": {"relevance":0,"actionability":0,"freshness":0,"reliability":0},
  "costs": {"tokens_est":120,"latency_est_ms":80,"dollars_est":0.0003},
  "risks": {"sensitivity":"low","leakage":0.02},
  "provenance": [{"cite":"...","hash":"..."}],
  "scope": {"tenant":"...","project":"...","user":"..."},
  "graph": {"entities":["..."],"relations":[...],"valid":[null,null]},
  "status": "raw|summarized|refined|skill"
}
```

DSPy integration plan

Treat each LM-dependent step as a DSPy program with: **Signature** → **Module** → **Metric** → **Optimizer** → **Trainset/Valset**.

Core DSPy signatures

1. **RankCandidates** – choose top-k items under budget with diversity.

python

 Copy code

```
class RankCandidates(dspy.Signature):
    """Select and order items that maximize task utility per token."""
    task_desc: str
    budget_tokens: int
    items_json: str # serialized candidates with features, costs, risks
    topk: int
    ranked_ids_json: str
```

- 2. SummarizeForSlot** – compress selected items for a named slot with faithfulness constraints.

python

 Copy code

```
class SummarizeForSlot(dspy.Signature):
    """Summarize items for a slot with budget and citation anchors."""
    slot_name: str
    max_tokens: int
    items_with_citations_json: str
    summary_text: str
    kept_citations_json: str
```

- 3. EVofDeferredQuery** – estimate expected value per unit latency to decide which live queries to execute.

python

 Copy code

```
class EVofDeferredQuery(dspy.Signature):
    """Predict EV and cost of executing a deferred query given task and hi
    task_desc: str
    query_plan_json: str
    ev_score: float
```

- 4. ConsolidateEpisode** – convert episodic logs to stable notes or skills with provenance.

python

 Copy code

```
class ConsolidateEpisode(dspy.Signature):
    """Distill episode into stable note/skill with provenance and deltas."
    episode_json: str
    prior_notes_json: str
    consolidation_json: str # typed inserts/updates
```

- 5. RedactForScope** – enforce scope and policy before sharing.

python

 Copy code

```
class RedactForScope(dspy.Signature):
    """Redact sensitive fields; output diff against original."""
```

```
target_scope: str
draft_json: str
policy_json: str
redacted_json: str
```

6. BlueprintFill – assemble the final prompt blueprint from slot fragments.

python

 Copy code

```
class BlueprintFill(dspy.Signature):
    """Produce a structured context blueprint with slot caps respected."""
    task_desc: str
    slot_specs_json: str
    fragments_json: str
    blueprint_json: str
```

Modules

Default to `dspy.Predict` for deterministic layout and `dspy.ChainOfThought` where internal reasoning improves calibration. Start simple; enable assertions for structural guarantees.

- `RankCandidates` → `dspy.Predict` + assertion that output IDs $\subseteq$ input IDs and $\text{length} \leq \text{topk}$.
- `SummarizeForSlot` → `dspy.ChainOfThought` with assertions on token count and citation coverage.
- `EVofDeferredQuery` → `dspy.Predict`; can be upgraded to a hybrid model that uses a small learned regressor as a feature.
- `ConsolidateEpisode` → `dspy.ChainOfThought`.
- `RedactForScope` → `dspy.Predict` with hard assertions on policy tags.
- `BlueprintFill` → `dspy.Predict`.

(Assertions in DSPy are a first-class way to encode output constraints without brittle prompting. Use them here.) Arize AI

Optimizers and metrics

Pick per-module metrics aligned to behavior and budget.

- **RankCandidates**
Metric: NDCG@k on human- or heuristic-labeled utility; penalty for redundancy and

risk violations.

Optimizer: **MIPROv2** when you have $\geq$ a few hundred labeled contexts; else **COPRO** to improve instructions quickly; add **BootstrapFewShotWithRandomSearch** to tune examples when labels are sparse. DSPy +2

- **SummarizeForSlot**

Metric: faithfulness judged by citation overlap + a constrained factuality scorer on extracted triples; hard cap on tokens.

Optimizer: **MIPROv2** for joint instruction + demo tuning; **COPRO** for instruction-only on small sets. DSPy +1

- **EVofDeferredQuery**

Metric: correlation between predicted EV and realized task reward delta; auxiliary loss on latency calibration.

Optimizer: **COPRO** to shape instruction; examples come from logged runs.

- **ConsolidateEpisode**

Metric: human approval rate on merges; reduction in future retrieval redundancy; provenance coverage.

Optimizer: **MIPROv2**.

- **RedactForScope**

Metric: zero policy violations on a red-team test set; minimal utility loss compared to unredacted baseline.

Optimizer: **COPRO**.

- **BlueprintFill**

Metric: downstream task success given same candidate set; enforce slot caps.

Optimizer: **MIPROv2**.

Background on these optimizers and their intended use appears in the official docs and write-ups. DSPy +3

Program compilation

Example skeleton that a coding agent can expand:

```
python
```

 Copy code

```
import dspy
```

```
# 1) LM backends
```

```
fast_lm = dspy.OpenAI(model="gpt-4o-mini") # cheap, fast
```

```

trust_lm = dspy.OpenAI(model="gpt-4.1")           # higher quality for summ

# 2) Modules
ranker      = dspy.Predict(RankCandidates)
summarizer  = dspy.ChainOfThought(SummarizeForSlot, lm=trust_lm)
ev_estimator= dspy.Predict(EVofDeferredQuery)
consolidate = dspy.ChainOfThought(ConsolidateEpisode, lm=trust_lm)
redactor    = dspy.Predict(RedactForScope)
composer    = dspy.Predict(BlueprintFill)

# 3) Optimizers
from dspy.teleprompt import MIPROv2, COPRO, BootstrapFewShotWithRandomSear

rank_opt    = MIPROv2(metric=ndcg_metric, auto="light")
sum_opt     = MIPROv2(metric=faithfulness_metric, auto="medium")
ev_opt      = COPRO(metric=ev_rank_correlation)
con_opt     = MIPROv2(metric=merge_quality_metric)
red_opt     = COPRO(metric=policy_safe_metric)
blue_opt    = MIPROv2(metric=downstream_success_metric)

# 4) Compile each module on its dataset
ranker      = rank_opt.compile(ranker, trainset=rank_train)
summarizer  = sum_opt.compile(summarizer, trainset=sum_train)
ev_estimator = ev_opt.compile(ev_estimator, trainset=ev_train)
consolidate = con_opt.compile(consolidate, trainset=con_train)
redactor    = red_opt.compile(redactor, trainset=red_train)
composer    = blue_opt.compile(composer, trainset=blue_train)

```

DSPy's GitHub and docs outline this compile flow and the role of signatures/modules.

GitHub +1

Memory service API for external agents

These are the only endpoints external agents call. Internals will invoke DSPy programs to deliver results.

- POST /candidates/propose → returns candidate set for a task state.
- POST /selector/select → returns selected IDs and a budget ledger.

- `POST /composer/assemble` → returns a structured blueprint for prompting.
- `POST /episodes/ingest` → logs actions/outcomes; returns consolidation diff.
- `POST /governance/redact` → returns redacted document and audit.
- `POST /skills/promote` → registers a reusable skill card with tests.

All requests include `task_desc`, `budgets` and `scopes`. All responses include `provenance` and `policy_audit`.

Selection and composition logic

Use a two-stage planner inside the selector:

1. **Query planning** – score `deferred_query` candidates with `EVofDeferredQuery` and execute only those with best EV per ms within latency budget.
2. **Set selection** – run `RankCandidates` on the merged pool, then pack into slots via `BlueprintFill`. Redact before returning to caller.

Objective is the submodular knapsack we discussed earlier, operationalized as DSPy-compiled modules so the scoring text is learned rather than hard-coded.

Datasets for compilation

Build small, purpose-specific trainsets from logs. Each example fits in a compact JSON row so it is context-cheap.

- `rank_train` – tuples of (`task_desc`, `items_json`, `budget_tokens`, `topk`, `ideal_order_ids`) produced from A/Bs or human curation.
- `sum_train` – (`slot_name`, `max_tokens`, `items_with_citations_json`, `target_summary_text`) with fact spans annotated.
- `ev_train` – (`task_desc`, `query_plan_json`, `realized_reward_delta`) from online runs.
- `con_train` – (`episode_json`, `prior_notes_json`, `gold_consolidation_json`) curated for correctness.
- `red_train` – (`target_scope`, `draft_json`, `policy_json`, `gold_redacted_json`) from policy fixtures.
- `blue_train` – (`task_desc`, `slot_specs_json`, `fragments_json`, `blueprint_json`) linked to downstream success labels.

This directly enables MIPROv2 and COPRO usage as documented. DSPy +1

Reference implementation plan

Milestone 1 – “Closed loop on single agent”

- Implement stores and the Candidate Generator.
- Add `DSPy RankCandidates` and `SummarizeForSlot` with COPRO first, then MIPROv2 when you have 200+ examples. DSPy
- Ship `/selector/select` and `/composer/assemble`.
- Metrics: precision@k, redundancy ratio, faithfulness score, latency budget hit rate.

Milestone 2 – “Live queries and governance”

- Add `EVofDeferredQuery` and the query-planning stage.
- Implement `RedactForScope` with policy fixtures and assertions.
- Metrics: EV calibration R, zero policy violations on test set.

Milestone 3 – “Consolidation and skill accrual”

- Add `ConsolidateEpisode` and nightly consolidation jobs; promote frequent plans into skill cards.
- Track skill win rate and reuse counts.

Milestone 4 – “Shared graph + multi-agent”

- Introduce the temporal graph store with streaming neighborhood updates.
- Add scope-aware subscriptions for selective sharing.
- Evaluate graph maintenance cost per update and temporal correctness rates.

Evaluation harness

- **Unit metrics per module** – NDCG@k, BERTScore/faithfulness, EV correlation, policy precision/recall.
- **End-to-end** – success per 1k tokens added, median latency, bloat ratio, staleness rate.
- **Ablations** – COPRO vs MIPROv2, with and without graph neighborhood features, with and without EV stage.

DSPy's optimizer pages describe inputs and typical configurations; mirror those in the harness to keep experiments comparable. DSPy +2

Context-efficiency tactics

- Keep train examples **JSON-first and minimal**, not long transcripts.
 - Store reusable slot fragments as short canonical snippets with IDs; feed IDs, not raw text, into summarizers that can dereference as needed.
 - Maintain **slot caps** and reserve token headroom for the model's own reasoning.
 - Prefer **COPRO** in early phases for small, cheap compiles; switch selective modules to **MIPROv2** when you accumulate data. DSPy +1
-

Risks and guardrails

- **Optimizer overfitting** – hold out realistic eval sets; rotate negative examples with near-duplicates to punish redundancy.
 - **Judger bias for faithfulness** – supplement LLM judges with exact citation overlap and triple-level checks.
 - **Latency blowups** – keep a strict latency budget ledger per request; the selector cannot exceed it.
 - **Scope leaks** – redaction runs before compose; assertions must fail closed.
-

References

- DSPy docs on signatures and modules. DSPy +1
 - DSPy overview and ICLR paper on compiling LM pipelines. DSPy +2
 - Optimizers – overview, MIPROv2, COPRO, BootstrapFewShotWithRandomSearch. DSPy +3
 - MIPRO technical details and empirical results. ACL Anthology +1
 - Practical intros and cheatsheet for usage patterns. DSPy +2
 - Assertions concept for guardrails. Arize AI
-

If you want this turned into a starter repo structure and thin Python stubs for each signature, I can lay out the files and unit-test harness so you can feed it straight to your coding agent.

         Sources

I think we need to invert or change some of the assumptions. Agents should be able to interact with this system proactively, but this system **MUST** be able to read from agent sessions and other data sources without relying on the agents to write data into this system. Similarly, this system must be able to actively inject context into these agents' context windows during their working sessions in a graceful way, without relying on the agents requesting this first (though they may actively request this, too).

As such, this system must be safe in its operation, resilient to failures, fast, and resource efficient.

Thought for 57s >

Absolutely. Here is a spec that assumes your memory system can 1) read agent sessions without their cooperation and 2) proactively inject context into live sessions, while staying safe, resilient, fast, and frugal.

Design overview

Build a **supervisor memory service** that sits beside all agents. It has two new powers:

1. **Observe** – tap read-only streams from agent sessions, tools, IDEs, and data backends.
2. **Inject** – deliver minimal, typed context patches into an agent's next turn through supported channels.

Internally, every LM-touching step is a DSPy program with a signature, metric, and optimizer, so context selection, compression, and safety checks are learned and measurable rather than prompt spaghetti. ICLR Proceedings +1

High level components

1. Session Tap

- Reverse-proxy or SDK hooks that mirror prompts, tool calls, code edits, and completions from agents and IDEs.
- Official hooks where available: OpenAI Realtime sessions for live context, Anthropic Computer Use for screen and action traces, Gemini function calling and session APIs for tool plans. You do not rely on agents to write into memory; you observe their IO. [OpenAI Platform](#) +5

2. Observer

- Canonicalizes session deltas into memory items, tags provenance, and extracts structured signals like task intent, entities, and constraints.
- Writes to a **bitemporal knowledge graph** plus a private per-agent working store. Use graph neighborhoods and community summaries for global context, not just vectors. [Microsoft GitHub](#) +1

3. Injector

- Applies a Context Patch Protocol (CPP) to deliver context in the smallest useful unit.
- Supports in-band channels (system prefix, tool result messages, function responses) and out-of-band UI nudges when the host allows it. With OpenAI's realtime and Anthropic's computer use, you can patch the very next turn; with Gemini APIs, inject via function results or system text. [OpenAI Platform](#) +3

4. Selector and Composer

- Knapsack + MMR over candidates under strict token, latency, and risk budgets.
- Slot-based blueprint, not a flat blob.
- Retrieval blends lexical, vector, and graph neighborhoods; global summaries only when needed. [Microsoft GitHub](#)

5. DSPy LM layer

- Each LM step is a DSPy program compiled with MIPROv2 or COPRO against local metrics. This replaces brittle prompting with trainable, audited behaviors.

[ICLR Proceedings](#) +2

6. Governance and Safety

- Policy labels, redaction transforms, consent scopes, and a risk budget per injection.

- Strict provenance and bitemporal validity for all facts. Inspired by Generative Agents' scoring, but extended with sensitivity and reliability. ACM Digital Library

Data model

Treat everything that could be injected as a **Context Candidate**:

yaml

 Copy code

```
id, kind: memory_note | tool_result | file_chunk | episode | plan | constr
content, source, timestamp
features: relevance, actionability, freshness, reliability, importance
costs: tokens_est, latency_est_ms, dollars_est
risks: sensitivity_label, leakage_prob
provenance: citations[]
scope: tenant, project, user
graph: entities[], relations[], valid: [t_start, t_end]
status: raw | summarized | refined | skill
```

Context Patch Protocol (CPP)

A patch is the minimal, typed unit of injected help. It decouples suggestion from transport.

yaml

 Copy code

```
Patch {
  id, task_id, target_session_id, ttl_ms
  slot: task_header | constraints | user_prefs | retrieved_evidence | tool
  body: text or structured payload
  budget: max_tokens, max_latency_ms
  policy: allowed_scopes[], sensitivity_max
  provenance: citations[]
}
```

Transports

- OpenAI – system prefix or tool result message in the next turn via Realtime or standard chat APIs.
- Anthropic – message content or file attachment, plus computer-use actions if enabled.
- Gemini – function call results, system text, or safety-routed attachments.

OpenAI Platform +4

Graceful delivery

- Only inject on one of: high EV per ms, safety concern, or explicit agent ask.
- Always small, typed, and cite sources. Never exceed reserved token headroom.

DSPy programs

Each signature has a metric and optimizer, making the system tunable and measured.

1. **ObserveSession** – infer task state and deltas from raw session IO.

Metric: intent extraction F1 and entity recall. Optimizer: COPRO. DSPy

2. **ProposePatches** – generate patch candidates from the current state and graph neighborhood.

Metric: downstream success uplift per 1k tokens. Optimizer: MIPROv2. DSPy

3. **RankCandidates** – order items under budgets with diversity.

Metric: NDCG@k with redundancy and risk penalties. Optimizer: MIPROv2. DSPy

4. **SummarizeForSlot** – compress into slot with citation coverage.

Metric: factuality with citation overlap and triple checks. Optimizer: MIPROv2.

ICLR Proceedings

5. **EVofDeferredQuery** – whether to run a live query before injecting.

Metric: correlation between predicted EV and realized reward delta. Optimizer: COPRO. DSPy

6. **RedactForScope** – enforce policy and scope.

Metric: zero violations on red-team set; minimal utility loss. Optimizer: COPRO.

DSPy

7. **BlueprintFill** – assemble final patch payloads.

Metric: token-cap compliance and downstream success. Optimizer: MIPROv2.

DSPy

DSPy gives you signatures, modules like Predict and CoT, and optimizers to compile on small trainsets from your own logs. DSPy

Selection and injection loop

1. **Ingest** – mirror session deltas, parse to candidates, update graph neighborhoods and private store.
2. **Screen** – compute quick EV for any deferred queries; run only a few with best EV per ms.
3. **Select** – greedy submodular with MMR under token, latency, and risk budgets.
4. **Compose** – fill slots with compressed, cited fragments.
5. **Redact** – fail closed on policy.
6. **Inject** – send one or more CPP patches via best transport for that session.
7. **Credit** – record outcomes and update per-module weights.

Generative Agents' relevance–recency–importance is a good starting signal, but you must add actionability, reliability, and cost to maximize efficacy per token.

ACM Digital Library

Safety, resilience, and performance

- **Safety and governance**
 - Risk budget per patch. Required provenance and time validity for all nontrivial claims.
 - Redaction before compose, not after. Audit every injection.
- **Resilience**
 - Queue the Observe and Inject paths separately with backpressure and circuit breakers.
 - Fallback modes: passive observe only, or local private store only if graph is degraded.
- **Performance**
 - Reserve headroom for the model's own reasoning.
 - Delta injection, not full rewrites.
 - Cache slot fragments and global community summaries. GraphRAG style global passes only on a slow cadence.

Microsoft GitHub

External API surface

- `POST /sessions/register` – declare a session tap with capabilities and transports.
- `POST /observe/ingest` – bulk session events.
- `POST /patches/propose` – return scored patches with budgets and risks.
- `POST /patches/inject` – send chosen patches via a specific transport and get an ack.
- `GET /patches/audit?session_id=...` – full provenance and policy log.

All responses include a budget ledger, citations, and policy verdicts.

Minimal transport adapters

- **OpenAI** – middleware for chat or Realtime that prepends a system patch or returns tool messages. OpenAI Platform +1
- **Anthropic** – message injection and, where allowed, computer-use actions to open files or notes that the agent instantly reads. Keep actions auditable and reversible. Anthropic
- **Gemini** – function call stubs that return structured context or tool args, plus system text for high level constraints. Google AI for Dev... +1

Reference build plan

Milestone 1 – single agent, passive observe and manual inject

- Session Tap as proxy, Observer, private store, basic selector and summarizer compiled with COPRO.
- Metrics: precision@k, bloat ratio, latency ledger.

Milestone 2 – proactive inject and safety

- Implement CPP, transports for one provider, RedactForScope, EVofDeferredQuery.
- Add DSPy MIPROv2 on Rank and Summarize once you have 200+ labeled examples. DSPy

Milestone 3 – temporal graph and community summaries

- Neighborhood updates and slow cadence global summaries, GraphRAG style, with incremental rebuilds.
- Metrics: temporal correctness and graph maintenance cost. Microsoft GitHub

Milestone 4 – multi-agent and skills

- Skill promotion with tests and win-rate tracking. Agents share vetted skills across sessions. This mirrors industry direction toward portable skills. The Verge

Evaluation

- **Efficacy per 1k tokens** – uplift in task success normalized by injected tokens.
- **Patch acceptance rate** – fraction of patches that change the next turn meaningfully.
- **Redundancy ratio** – percent of injected tokens that add no new facts.
- **Staleness** – fraction of facts past half life.
- **Latency SLA** – p95 and p99 for observe to inject.
- **Safety** – policy violations per 10k injections, all must be zero in pre-prod.

Threat model

- **Silent context poisoning** – mitigate with provenance and risk budgets.
- **Transport failure** – fall back to passive mode with alarms.
- **Over-injection** – hard caps, A/B throttle, and per-session cool-downs.
- **Privacy leaks** – scoped reads, redaction, and explicit consent registry.

Why this meets your new constraints

- The system reads sessions through taps and proxies. Agents do not need to write anything.
- Proactive context injection happens via CPP and provider-specific transports, without an agent requesting it.
- Safety is enforced pre-compose and pre-inject, with strong provenance and policy budgets.
- Resilience is designed in via decoupled observe and inject paths, circuit breakers, and fallbacks.

- Efficiency is maximized with EV-gated queries, submodular selection, slot blueprints, deltas, and DSPy-compiled summarizers.

Key references

- DSPy overview and ICLR paper, signatures, modules, and optimizers. [DSPy](#) +3
- OpenAI Realtime and API references for live session integration. [OpenAI Platform](#) +1
- Anthropic Computer Use capability and announcement. [Anthropic](#) +1
- Gemini function calling and API references. [Google AI for Dev...](#) +1
- GraphRAG for global summaries and hierarchical retrieval. [Microsoft GitHub](#) +2
- Generative Agents memory scoring for relevance, recency, importance.

[ACM Digital Library](#) +1

If you want, I can turn this into a repo scaffold with CPP adapters for one provider first and DSPy stubs for each signature, plus a harness to log and label examples for MIPROv2.

        Sources

create a downloadable zip that contains PRD.md, SPEC.md, ARCHITECTURE.md, RESEARCH-REFERENCES.md, and whatever code, data, scripts, etc would be valuable in making this an effective starting point for an agent or human to build from. make sure you review this entire session when building this, as i don't want you to drop key context.

Thought for 1m 14s >

All set. Here's your starter package zipped up with PRD.md, SPEC.md, ARCHITECTURE.md, RESEARCH-REFERENCES.md, and a working scaffold of code, data, and scripts.

[Download the zip](#)

It includes:

- A uv-friendly Python project with DSPy signatures and stubs, a Context Patch Protocol, selection and retrieval skeletons, governance hooks, and transport adapters for OpenAI, Anthropic, and Gemini.
- FastAPI dev server, tiny train datasets for optimizer bootstrapping, and basic tests.
- PRD/SPEC/ARCHITECTURE docs aligned to the proactive observe-and-inject model, plus a curated references file with verified links.

If you want, I can also generate a minimal demo that simulates a live agent session and shows a context patch being injected via the OpenAI adapter stub.

</>



the system should be named gyre. rename it and give me a new download.

done. your project is now named Gyre and packaged fresh.

[Download the new zip](#)

Highlights

- Repo renamed and refactored to gyre with updated package imports.
- Docs and server titles updated to Gyre.
- Env vars switched to GYRE_... keys.
- Same uv-friendly scaffold, transports, DSPy signatures, and dev server.

//